AIOT Android awCamera API
使用指南

版本号: 1.3

发布日期: 2025.09.29

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.07.25	KPA0535	初始版本
1.1	2024.07.29	KPA0535	支持 6 路摄像头; 替换 360 算法
1.2	2025.04.29	KPA0573	适配 A733 T736
1.3	2025.09.29	AWA1739	适配 A537 A333

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1
1.4 相关术语 1

2 模块介绍 **3**
2.1 模块功能介绍 3
2.2 模块配置介绍 3

3 接口定义 **5**
3.1 打开摄像头 5
3.1.1 open(int startId, int cameraNum, int width, int height) 5
3.1.2 open(int startId, int cameraNum, int width, int height, int type) 5
3.2 预览 6
3.2.1 startPreview 6
3.2.2 stopPreview 6
3.2.3 startRender 6
3.2.4 stopRender 7
3.2.5 setAWCameraPreviewMax 7
3.2.6 awCamSetViewOrientation 7
3.2.7 setAWDisplayOrientation 8
3.2.8 setAWDisplayPreviewType 8
3.2.9 setAWCameraPreviewPosition 8
3.2.10 setParameters 9
3.3 拍照 10
3.3.1 setParameters 10
3.3.2 takepicture 10
3.4 录像 11
3.4.1 awCamRecInit 11
3.4.2 awCamRecStart 11
3.4.3 awCamRecStop 11
3.4.4 awCamRecRelease 12
3.4.5 awCamRecStartNextFile 12
3.4.6 awCamRecInit（支持子码流） 12
3.4.7 awCamSubRecStart 13
3.4.8 awCamSubRecStop 13
3.4.9 awCamSubRecGetSpspps 14
3.4.10 awCamSetShareBuffer 14

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4.11 awCamSetShareBuffer（支持缩放） . . . 15
3.4.12 awCamRecSetParaAndCmds . . . 15
3.4.13 awCamRecGetParaAndCmds . . . 15
3.4.14 awCamRecSetAudioChannelIndex . . . 16
3.5 水印 . . . 16
3.5.1 startWaterMark . . . 16
3.5.2 stopWaterMark . . . 16
3.5.3 setWaterMarkMultiple . . . 17
3.5.4 setWaterMarkMultiple(String mWaterMark,int dispMode) . . . 17
3.5.5 awCamSetPicturePath(String fileName) . . . 17

4 示例 18
4.1 录制视频封装格式 . . . 18
4.2 音频模式 . . . 18
4.3 拍照示例 . . . 18
4.3.1 默认保存文件到本地 . . . 18
4.3.2 回调 YUV 数据 . . . 19
4.3.3 回调 Jpeg 数据且不写入到文件 . . . 19
4.4 YUV 数据回调示例 . . . 20
4.5 大小码流录像示例 . . . 20
4.5.1 主码流录文件 ts + 子码流 h264 回调 + 录音 . . . 20
4.5.2 仅主码流录文件 ts + 录音（无子码流） . . . 21
4.5.3 仅子码流 h264 回调 + 录音 . . . 21
4.5.4 缩时录影 . . . 21
4.5.5 多通道录音 . . . 21

5 FAQ 23
5.1 摄像头无法打开或打开后 hal 层报错 select timeout . . . 23
5.2 打开摄像头预览出现黑屏现象 . . . 23
5.3 预览/录像出现上/下边缘显示绿条 . . . 24
5.4 Camera Hal 是否支持 Hal 3.4 . . . 24
5.5 如何后台抓取 log . . . 24

广州星翼电子科技有限公司zdyzlinux

1 前言

1.1 编写目的

基于车载业务的特点，考虑到客户的定制化需求和 Camera 使用的性能要求，于是在 Android 原生 Camera API1.0 框架的基础上，增加了一系列 DVR 相关的接口，命名为 awCamera，源码路径位于 frameworks/av/services/camera/libcameraservice/awcamera。

开发者可参考 SDK 中 DVR 应用，并根据 Camera.java 提供的接口，以及参考本文档的介绍，在 Android 应用程序中，通过反射等调用方法，实现 DVR 预览、拍照、录制、获取 YUV 裸流、H264 码流数据等功能。

1.2 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	Android 版本
T527	Linux-5.15	Android13
T527V	Linux-5.15	Android13
A733	Linux-5.15	Android13
T736	Linux-5.15	Android13
A537	Linux-5.15	Android13
A333	Linux-5.15	Android13

1.3 相关人员

Camera 模块开发人员、应用开发人员。

1.4 相关术语

版权所有 © 珠海全志科技股份有限公司。保留一切权利

术语	说明
bitrate	录像码率。
fps	录像帧率。
ts/mp4	视频文件的封装容器。
sps/pps	H264 的码流数据的头信息。
四方位预览	同时显示 4 路 camera 画面，应用可指定任意位置拼接显示。
大小码流录像	主码流默认封装并保存至本地，子码流（H264/AAC）默认回调至应用端。
预分配	为解决长时间录像导致的磁盘碎片化问题，引入预分配机制，目前仅支持
	FAT32 格式 U 盘。
分段录制	每隔 1/3/5 分钟封装成一个视频文件，切换期间不丢帧。
多通道录音	多个 MIC 采集数据后经驱动合成一路多通道数据，应用层拆分不同通道
	数据后封装到指定的文件中。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

可通过调用本文档的接口，使用 Android 平台的一个或多个摄像头，进行预览画面、拍照以及录像功能。

当前最新版本：

libawcamera 的 logcat 关键字：AWCameraInterface VER: 20240315-v3.00

libAWCamRecorder 的 logcat 关键字：f: AWCamRecorder,l:163,ver: 20220915-v2.83。

已支持的功能包含：

```
1.支持单路预览/拍照/录像；
2.支持二合一（左右/上下拼接）预览/拍照/录像；
3.支持四合一（四宫格）预览/拍照/录像；
4.支持两路独立录像；
5.支持四路独立录像；
6.支持四方位预览/拍照/录像；
7.支持大小码流录像；
8.支持缩时录影功能；
9.支持水印功能；
10.支持录像预分配功能；
11.支持分段录制；
12.支持多通道录音；
13.支持多路合成可自定义Camera画面位置。
```

2.2 模块配置介绍

以下为选择 awCamera 接口的配置：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Android 13: device/softwinner/saturn/txxxxxxx/device-common.mk
USE_CAMERA_HAL_3_4 ?= false
USE_CAMERA_HAL_AUTO_1_0 ?= true
```

配置说明：

- Android Camera 原生接口使用 USE_CAMERA_HAL_3_4 宏定义。
- awCamera 框架接口使用 USE_CAMERA_HAL_AUTO_1_0 宏定义，用于开发车载应用。

注意

若调用 *awCamera* 的框架接口，但是 *Hal* 选择了 *USE_CAMERA_HAL_1_0*，将会导致预览黑屏，录像失败等现象。

广州星翼电子科技有限公司zdyzlinux

3 接口定义

3.1 打开摄像头

3.1.1 open(int startId, int cameraNum, int width, int height)

```
Camera open(int startId, int cameraNum, int width, int height)
```

1. 作用：打开摄像头，获取 Camera 对象。
2. 参数：
   - startId：/dev/videoX 的节点起始序列号。
   - cameraNum：顺序打开的 Camera 数目，支持 1/2/4/6。
   - width/height：每个摄像头预览的宽高。
3. 返回值：Camera 对象。
4. 调用说明：cameraNum 大于 1 时，需要保证每个摄像头的分辨率一致，否则无法合成显示。

3.1.2 open(int startId, int cameraNum, int width, int height, int type)

```
Camera open(int startId, int cameraNum, int width, int height, int type)
```

1. 作用：打开摄像头，获取 Camera 对象。
2. 参数：
   - startId：/dev/videox 的节点起始序列号。
   - cameraNum：顺序打开的 Camera 数目，支持 1/2/4。
   - width/height：每个摄像头预览的宽高。
   - type：0 表示固定位置拼接；1 表示 360 环视拼接 (已废弃)；2 表示四方位拼接；3 表示固定位置拼接。
3. 返回值：Camera 对象。
4. 调用说明：
   - type 为 0 时，cameraNum 支持 1/2/4，表示 cameraNum 路 Camera 合成显示，由 G2D 合成显示，录像/拍照画面与预览画面保持一致。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- type 为 1 时，cameraNum 需为 4，表示 360 环视场景，公版使用第三方 360 3D 算法库预览，录像/拍照画面均为四宫格画面。（已废弃）
- type 为 2 时，cameraNum 需为 4，表示四方位场景，由 GPU 合成显示，预览画面的位置可使用 setAWCameraPreviewPosition 方法自由设置，录像为固定的四宫格画面，拍照默认保存前视画面。
- type 为 3 时，cameraNum 需为 4，表示经 GPU Scale 后的四宫格场景，预览画面位置大小不可修改，录像/拍照画面均为四宫格画面。
- type 为 0 与 3 的区别：0 是使用 G2D 模块进行合成显示，3 是使用 GPU 模块进行合成显示。建议使用 type0，type3 可做测试使用。
- cameraNum 大于 1 时，需要保证每个摄像头的分辨率一致，否则无法合成显示。
- 四路拼接模式下配合调用标准接口 parameters.set(“preview-360birdview”,1); 可以进入 360 3D 算法预览

3.2 预览

3.2.1 startPreview

```
void startPreview()
```

1. 作用：开启 Camera 系统的数据采集工作以及前台预览。
2. 参数：无。
3. 返回值：无。
4. 调用说明：开启 Camera 数据的采集功能。

3.2.2 stopPreview

```
void stopPreview()
```

1. 作用：停止 Camera 系统的数据采集工作以及前台预览。
2. 参数：无。
3. 返回值：无。
4. 调用说明：停止 Camera 数据的采集功能，调用此功能也会停止录像功能。

3.2.3 startRender

```
void startRender()
```

1. 作用：仅开启 Camera 系统的前台预览。
2. 参数：无。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3. 返回值：无。
4. 调用说明：前提需调用 startPreview 才能生效，且只对预览生效，不影响后台数据流。

3.2.4 stopRender

```
void stopRender()
```

1. 作用：仅停止 Camera 系统的前台预览，系统切换至后台录像。
2. 参数：无。
3. 返回值：无。
4. 调用说明：

- 当开启 startPreview 之后，系统切换到后台录像时调用此接口。
- 后续可与 startRender 方法搭配使用。
- stopRender 不会停止后台数据流的采集。

3.2.5 setAWCameraPreviewMax

```
void setAWCameraPreviewMax(int cameraIndex)
```

1. 作用：最大化二/四合一场景的某一路画面，全屏显示。
2. 参数：
   - cameraIndex：camera 相对 ID。
   - 二合一显示时，cameraIndex 取值 1/2，1 表示第 1 路画面最大化显示。
   - 四合一显示时，cameraIndex 取值 1/2/3/4，1 表示第 1 路画面最大化显示。
   - cameraIndex 为 0 时恢复默认合成画面。
3. 返回值：无。
4. 调用说明：仅在二/四合一场景下有效，最大化仅对预览/拍照画面有效，对录像画面无效。

3.2.6 awCamSetViewOrientation

```
void awCamSetViewOrientation(int index, int degrees, int mirror)
```

1. 作用：控制某一个摄像头的画面进行旋转与左右镜像显示。
2. 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- index：该参数为相对值，与 CameraID 无关，即第一路 Camera 的 index 为 0，第二路的 index 为 1，以此类推。
- degrees：旋转度数，只支持 90 的整数倍，即 0/90/180/270。
- mirror：水平翻转，1 为水平翻转，0 为不翻转。

3. 返回值：无。
4. 调用说明：无。

3.2.7 setAWDisplayOrientation

```
void setAWDisplayOrientation(int degrees,int isMirror)
```

1. 作用：Android 原生的镜像旋转，对整个 SurfaceView 有效。
2. 参数：
   - degrees：旋转度数，只支持 90 的整数倍，即 0/90/180/270。
   - mirror：水平翻转，1 为水平翻转，0 为不翻转。
3. 返回值：无。
4. 调用说明：
   - 不建议使用，推荐使用 awCamSetViewOrientation。
   - Android 12 上此接口无效，推荐使用 awCamSetViewOrientation。

3.2.8 setAWDisplayPreviewType

```
void setAWDisplayPreviewType(int type)
```

1. 作用：设置二合一画面的显示类型。
2. 参数：type：0：左右合成显示，1：上下合成显示。
3. 返回值：无。
4. 调用说明：仅在二合一场景下生效，对预览画面有效，对拍照/录像画面无效。

3.2.9 setAWCameraPreviewPosition

```
void setAWCameraPreviewPosition(int cameraId,int left,int top,int width,int height,int sync)
```

1. 作用：设置四方位摄像头预览位置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 参数：
   - cameraId：camera 相对 ID，可设置的摄像头 ID 为 0/1/2/3，0 表示第 1 路摄像头画面。
   - left/top: 对应 cameraId 下画面的起始点坐标。
   - width/heigth：对应 cameraId 下画面从起始点坐标开始的宽度/高度。
   - sync：对应 cameraId 下画面的修改是否立即生效，1：立即生效，0：重启预览后生效。
3. 返回值：无。
4. 调用说明：预览画面可任意修改位置，拍照固定保存前视图，录像固定保存四宫格画面。

3.2.10 setParameters

```
void setParameters(Parameters params)
```

1. 作用：多路合成场景下前后左右摄像头位置调整。
2. 参数：
   - params：Camera 属性，继承原生接口。
   - params：新增接口包含 camera-direct-front/camera-direct-back/camera-direct-left/camera-direct-right 四个属性。
   - camera-direct-front：前视图的相对 ID，取值范围：[0,5]，0 表示第 1 路摄像头。
   - camera-direct-back：后视图的相对 ID，取值范围：[0,5]，1 表示第 2 路摄像头。
   - camera-direct-left：左视图的相对 ID，取值范围：[0,5]，2 表示第 3 路摄像头。
   - camera-direct-right：右视图的相对 ID，取值范围：[0,5]，3 表示第 4 路摄像头。
   - camera-direct-inner 扩展视图 1 的相对 ID, 取值范围：[0,5]，4 表示第 5 路摄像头。
   - camera-direct-bottom 扩展视图 2 的相对 ID, 取值范围：[0,5]，5 表示第 6 路摄像头。
   - i-frame-interval: 录像 I 帧间隔设置，默认值为 30
   - preview-frame-rate：setFullPreviewFps 接口在 libawcamera.so 的版本 v2.90 中已经弃用，预览帧率，区别于原生接口的帧率设置，此处帧率实现方式不是通过驱动，而是通过丢帧方式实现的帧率设置，驱动出图帧率不受影响
   - preview-360birdview: 车载全景 360 3D 模式预览
3. 回值：无。
4. 用说明：

- 若只需要设置预览位置相关的属性，只需设置相应的属性，其他属性将保留上次设置的值。
- 在四合一场景下，若希望设置前视图画面为第 3 路摄像头画面，可设置参数 parameters.set(“camera-direct-front”,2)。
- 在二合一场景，若希望前后视图反过来，可设置参数 parameters.set(“camera-direct-front”,1);parameters.set(“camera-direct-back”,0)。
- 此接口功能在 libawcamera.so 的 20220425-v2.82 及以后版本中支持。
- 在 libawcamera.so 的 20221103-v2.90 版本后四方位预览与录像调整为一致顺序

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3 拍照

3.3.1 setParameters

```
void setParameters(Parameters params)
```

1. 作用：设置 jpeg 照片路径，照片叠加水印功能。
2. 参数：
   - params：Camera 属性，继承原生接口。
   - params：新增接口包含 picture-watermark。
   - picture-watermark：回调的 YUV 数据是否需要包含水印。
3. 回值：无。
4. 用说明：

- 若只需要设置拍照相关的属性，请只设置相应的属性即可，其中若需要叠加水印时 picture-watermark 属性需要每次都设置。
- 当 picture-watermark 属性为 “true”，或为任意值时，回调的 YUV 数据包含水印显示。
- 当 picture-watermark 属性为 NULL ，或不设置该属性时，回调的 YUV 数据不含水印显示。

3.3.2 takepicture

```
void takePicture(ShutterCallback shutter, PictureCallback raw, PictureCallback jpeg)
```

1. 作用：触发拍照，保存 jpg 文件到本地。
2. 参数：
   - shutter：快门回调。
   - raw：图像 raw 数据回调。
   - jpeg：图像 jpeg 数据回调。
3. 返回值：无。
4. 调用说明：目前支持 shutter/raw/jpeg 回调，使用方法请参考拍照示例。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4 录像

3.4.1 awCamRecInit

```
void awCamRecInit(String filename, int format, int width, int height, int framerate, int
    bitrate, int audiomode, int audioSampleRate)
```

1. 作用：录制初始化。
2. 参数：
   - filename：录制文件绝对路径与名称。
   - format：封装格式，支持 ts 或 mp4 格式，见 AwCamRecFileFormat 定义录制视频封装格式。
   - width/height：编码输出的视频宽高。
   - framerate：录像帧率，最高支持 25fps，最低 1fps。
   - bitrate：录像码率，如 2M，需设置为 20000000。
   - audiomode：使能录音，1 为开启录音，0 为不开录音。
   - audioSampleRate：音频的采样率。
3. 回值：无。
4. 用说明：
   - 录制前需要使用该接口设置录制参数。
   - 录像配置的 fps 将会同步修改预览的 fps，若期望预览 fps 不变，可在调用此接口之前设置属性：setprop persist.sys.camera.preview.fullfps “0xFF”。
   - persist.sys.camera.preview.fullfps 属性支持需保证 libawcamera.so 版本号不低于 20220704-v2.88。

3.4.2 awCamRecStart

```
void awCamRecStart()
```

1. 作用：启动录制。
2. 参数：无。
3. 返回值：无。
4. 调用说明：调用之前需先调用 awCamRecInit()。

3.4.3 awCamRecStop

```
void awCamRecStop()
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1. 作用：停止录制。
2. 参数：无。
3. 返回值：无。
4. 调用说明：当需要完全停止录像时需要继续调用 awCamRecRelease() 以释放资源。

3.4.4 awCamRecRelease

```
void awCamRecRelease()
```

1. 作用：释放录制资源。
2. 参数：无。
3. 返回值：无。
4. 调用说明：调用 awCamRecStop 后需要再调用 awCamRecRelease 接口释放资源。

3.4.5 awCamRecStartNextFile

```
void awCamRecStartNextFile(String filenane)
```

1. 作用：分段录制功能，切换到下一个录像文件。
2. 参数：无。
3. 返回值：无。
4. 调用说明：设置下一个录制文件的名称，即将保存到这个文件，前面录制的文件将关闭，支持数据不丢帧存储。

3.4.6 awCamRecInit（支持子码流）

```
void awCamRecInit(String filename, int format, FileDescriptor fileDescriptor, int memLen, int
     offset, int width0, int height0, int frameRate0, int bitRate0, int width1, int height1,
    int frameRate1, int bitRate1, int audioMode, int audioSource, int audioSampleRate,int
    audioChannel, int timeLapse)
```

1. 作用：大小码流录制初始化。
2. 参数：

   - filename：录制文件绝对路径与名称。
   - format：封装格式，支持 ts 或 mp4 格式，见 AwCamRecFileFormat 定义录制视频封装格式。
   - fileDescriptor：sharebuffer 的 fd 句柄，用于回调 H264 码流数据。
   - memLen：sharebuffer 的总内存大小，为偏移长度 + H264 码流数据长度。
   - offset：sharebuffer 的头部偏移长度。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- widthX/heightX：大小码流的编码输出的视频宽。
- frameRateX：大小码流的帧率，最高支持 25fps，最低 1fps。
- bitrate：录像码率，如 2M，设置为 20000000。
- audioMode：使能录音，1 为开启录音，0 为不开录音。
- audioSampleRate：音频的采样率，一般设置 16000Hz。
- audioChannel：录音采样通道数，最大支持 16 通道录音，1 表示单通道录音，2 表示立体声录音，3~16 表示当前场景具有多少个通道数据。
- timeLapse：是否使能缩时录影，0 表示禁用缩时录影，N 表示每 Ns 取一帧数据，默认保存视频帧率为 frameRate0 fps。

3. 回值：无。
4. 用说明：

- 录制前需要使用该接口设置录制参数，支持单独主码流或子码流录像，具体使用可参考大小码流录像示例。
- 录像配置的 fps 将会同步修改预览的 fps，若期望预览 fps 不变，可在调用此接口之前设置属性：setprop persist.sys.camera.preview.fullfps “0xFF”。
- persist.sys.camera.preview.fullfps 属性支持需保证 libawcamera.so 版本号不低于 20220704-v2.88。

3.4.7 awCamSubRecStart

```
void awCamSubRecStart(int videoCh,int audioEn)
```

1. 作用：单独启动主码流或者子码流录制。
2. 参数：

- videoCh：0 代表仅开启主码流录像，1：代表仅开启子码流录像。
- audioEn：是否开启录音，1 表示开启录音。

3. 返回值：无。
4. 调用说明：支持先开启主码流录像，再开启子码流录像，但是需要初始化两路录像的参数。

3.4.8 awCamSubRecStop

```
void awCamSubRecStop(int videoCh)
```

1. 作用：单独停止主码流或者子码流录制。
2. 参数：videoCh：0 代表仅停止主码流录像，1：代表仅停止子码流录像。
3. 返回值：无。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4. 调用说明：

- awCamSubRecStop 与 awCamSubRecStart 可搭配使用。
- 当不再需要录像时依旧需要调用 awCamRecRelease() 以释放资源。

3.4.9 awCamSubRecGetSpspps

```
int[] awCamSubRecGetSpspps(int videoCh)
```

1. 作用：获取码流的 SPS/PPS 信息。
2. 参数：videoCh：0：主码流，1：子码流。
3. 返回值：返回 SPS/PPS 信息，长度一般不超过 64 字节。
4. 调用说明：

- 当仅开启 H264 码流回调时，videoCh 设置为 0。
- 当开启主码流录像及子码流回调时，videoCh 设置为 1。

3.4.10 awCamSetShareBuffer

```
int awCamSetShareBuffer(FileDescriptor fileDescriptor, int cameraId, int memLen, int offset)
```

1. 作用：设置共享内存文件描述符，用于获取 yuv 数据。
2. 参数：

- fileDescriptor：sharebuffer 的 fd 句柄，用于回调 YUV 数据。
- cameraId：与 OpenCamera 的 CameraId 一致，若为多合一场景，则为起始 ID。
- memLen：sharebuffer 的总内存大小，为偏移长度 + YUV 数据长度。
- offset：sharebuffer 的头部偏移长度。

3. 返回值：

- 成功：0。
- 失败：-1。

4. 调用说明：默认获取 1280x720 分辨率大小，帧率为 25fps 的 YUV 数据。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4.11 awCamSetShareBuffer（支持缩放）

```
int awCamSetShareBuffer(FileDescriptor fileDescriptor, int cameraId, int memLen, int offset,
    int width, int height, int fps)
```

1. 作用：设置共享内存文件描述符，用于获取 yuv 数据，支持缩放/自定义帧率功能。
2. 参数：
   - fileDescriptor：sharebuffer 的 fd 句柄，用于回调 YUV 数据。
   - cameraId：与 OpenCamera 的 CameraId 一致。
   - memLen：sharebuffer 的总内存大小，为偏移长度 + YUV 数据长度。
   - offset：sharebuffer 的头部偏移长度。
   - width/height：回调的 YUV 数据的宽/高，支持与原始数据不一致。
   - fps：yuv 数据回调的帧率。
3. 返回值：
   - 成功：0。
   - 失败：-1。
4. 调用说明：参数 width 或 height 与 opencamera 的 w 与 h 参数不一致时，会自动缩放到 width x height 的尺寸。

3.4.12 awCamRecSetParaAndCmds

```
void awCamRecSetParaAndCmds(String parameters,int cmds)
```

1. 作用：设置录像参数，此接口已弃用。
2. 参数：无。
3. 返回值：无。
4. 调用说明：无。

3.4.13 awCamRecGetParaAndCmds

```
String awCamRecGetParaAndCmds(int cmds)
```

1. 作用：获取录像参数，此接口已弃用。
2. 参数：无。
3. 返回值：无。
4. 调用说明：无。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4.14 awCamRecSetAudioChannelIndex

```
void awCamRecSetAudioChannelIndex(int index)
```

1. 作用：在多通道录音场景下，绑定对应的通道 index 进行录音。
2. 参数：index：多通道中的第 index 路通道，index 从 0 开启计数。
3. 返回值：无。
4. 调用说明：仅在调用 awCamRecInit（支持子码流）接口时且参数 audioChannel 为 3~16 时有效，当 audioChannel 取值为 16 时，index 取值范围为 0~15，表示提取 ch0~ch15 中其中一个通道的数据进行封装。

3.5 水印

3.5.1 startWaterMark

```
void startWaterMark()
```

1. 作用：开启水印叠加显示功能。
2. 参数：无。
3. 返回值：无。
4. 调用说明：
   - 开启后，预览/拍照/录像画面默认会显示时间水印，支持预览画面不显示水印。
   - 预览画面不显示水印设置方法：setprop persist.sys.camera.preview.watermark “0x00”。
   - 预览画面显示水印设置方法：setprop persist.sys.camera.preview.watermark “0xFF”。

3.5.2 stopWaterMark

```
void stopWaterMark()
```

1. 作用：关闭水印叠加显示功能。
2. 参数：无。
3. 返回值：无。
4. 调用说明：调用后会关闭所有场景下的水印显示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.5.3 setWaterMarkMultiple

```
void setWaterMarkMultiple(String mWaterMark)
```

1. 作用：自定义设置字符串水印, 最大不超过 20 个字符。
2. 参数：mWaterMark：字符水印，支持大写字母和数字，以及三个特殊字符，分别为'/'和';'和'-'。
3. 返回值：无。
4. 调用说明：字符串水印需要和坐标拼接，才能正常显示，即 mWaterMark 的实际内容为："0,0,0,10,100," + waterMark。

3.5.4 setWaterMarkMultiple(String mWaterMark,int dispMode)

```
void setWaterMarkMultiple(String mWaterMark,int dispMode)
```

1. 作用：自定义设置字符串水印。
2. 参数：
   - mWaterMark：字符水印。
   - dispMode：显示模式，该参数不起作用，设置为 0。
3. 返回值：无。
4. 调用说明：此接口与 setWaterMarkMultiple(String mWaterMark) 作用一致。

3.5.5 awCamSetPicturePath(String fileName)

```
void awCamSetPicturePath(String fileName)
```

1. 作用：设置照片输出路径。
2. 参数：
   - fileName：字符水印。
3. 返回值：无。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 示例

4.1 录制视频封装格式

```
 public static class AwCamRecFileFormat {
        public static final int H264_TS = 0;
        public static final int H265_TS = 1;
        public static final int H264_MP4 = 2;
        public static final int H265_MP4 = 3;
}
```

awcamera 框架下目前仅支持 H264_TS 与 H264_MP4 两种封装格式。

4.2 音频模式

```
public static class AwCamRecAudioMode {
        public static final int OFF = 0;
        public static final int NORMAL = 1;
}
```

在 awCamRecInit 方法中 audioMode 参数的取值，如 AwCamRecAudioMode 描述，主要有 OFF 和 NORMAL 两种类型。

4.3 拍照示例

4.3.1 默认保存文件到本地

第一步：先调用 awCamSetPicturePath 方法，设置保存的照片路径与文件名。

```
awCamSetPicturePath(path);
```

第二步：然后调用拍照方法，触发拍照，若保存成功，会将结果回调给 Camera.ShutterCallback。

```
Camera mCamera;
awCamSetPicturePath(path);
mCamera.takePicture(Camera.ShutterCallback, null, null);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3.2 回调 YUV 数据

第一步：先调用 setParameters 方法，设置保存的照片路径与文件名。

若没有调用此方法，则会覆盖上次传递的路径，初始默认路径为/mnt/sdcard/test.jpg。

若需要 YUV 增加水印（水印类型与录像水印一致），则可设置 picture-watermark 属性，当不设置或设置为 NULL 时，YUV 数据不含水印。

```
Camera.Parameters parameters = mCamera.getParameters();
parameters.set("picture-watermark", "true");
mCamera.setParameters(parameters);
```

第二步：然后设置回调数据的 Buffer 大小，Buffer 大小需与合成后的图像大小一致，如 4 路 720p 四合一场景下的 size 计算方法如下：

```
int mBufferSize = (1280 * 2 * 720 * 2) * 3 / 2;
byte[] mRawBuffer = new byte[mBufferSize];
Camera mCamera;
mCamera.addRawImageCallbackBuffer(mRawBuffer);
```

第三步：最后调用拍照方法，触发拍照，若保存成功，则会将结果回调给 Camera.ShutterCallback，同时把 YUV 数据回调给 Camera.PictureCallback。

```
Camera mCamera;
mCamera.takePicture(Camera.ShutterCallback, Camera.PictureCallback, null);
```

更详细的使用方法可参考 SDK 中 DVR 应用：android/vendor/aw/public/package/apk/DVR。

4.3.3 回调 Jpeg 数据且不写入到文件

第一步：然后设置回调数据的 Buffer 大小，Buffer 大小需与合成后的图像大小一致，如 4 路 720p 四合一场景下的 size 计算方法如下：

```
int mBufferSize = (1280 * 2 * 720 * 2) * 3 / 2;
byte[] mRawBuffer = new byte[mBufferSize];
Camera mCamera;
mCamera.addRawImageCallbackBuffer(mRawBuffer);
```

第二步：最后调用拍照方法，触发拍照，若保存成功，则会将结果回调给 Camera.ShutterCallback，同时把 Jpeg 数据回调给 Camera.PictureCallback。

```
Camera mCamera;
mCamera.takePicture(Camera.ShutterCallback, null, Camera.PictureCallback);
```

更详细的使用方法可参考 SDK 中 DVR 应用：android/vendor/aw/public/package/apk/DVR。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.4 YUV 数据回调示例

第一步：申请一块共享内存。

```
int mCameraWidth = 1280;
int mCameraHight = 720;
int mOffsetShareBufferSize = 20;
int mMemSize = mCameraWidth * mCameraHight * 3 / 2 + mOffsetShareBufferSize;
MemoryFile mMemoryFile = new MemoryFile("com.softwinner.dvr", mMemSize);
```

第二步：初始化共享内存，设置读写标志位为可写，默认 byte0 为读写标志位，0 代表可读，1 代表可写。

```
byte[] mShareBuffer = new byte[mMemSize];
for(int i = 0; i < mShareBuffer.length; i++)
    mShareBuffer[i] = 0;
mShareBuffer[0] = 1;
mMemoryFile.writeBytes(mShareBuffer, 0, 0, mShareBuffer.length);
```

第三步：设置共享内存给 Camera 对象，并绑定对应的 CameraID，若为多合一场景，则为起始 ID。

```
Camera mCamera;
mCamera.setMemoryFileFileDescriptor(mMemoryFile.getFileDescriptor(), mCameraID, mMemSize,
    mOffsetShareBufferSize);
```

第四步：最后监听 MemoryFile 的读写标志位，及时获取 YUV 数据，并设置读写标志位。

```
byte[] mOffsetShareBuffer = new byte[mOffsetShareBufferSize]
mMemoryFile.readBytes(mOffsetShareBuffer, 0, 0, mOffsetShareBufferSize);
if (mOffsetShareBuffer[0] == 0) {
    mMemoryFile.readBytes(mShareBuffer, mOffsetShareBufferSize, 0, mShareBuffer.length);
    mShareBuffer[0] = 1;
    mMemoryFile.writeBytes(mShareBuffer, 0, 0, 1);
}
```

更详细的使用方法可参考 SDK 中 DVR 应用：android/vendor/aw/public/package/apk/DVR，若出现丢帧等现象，可以缩小每次获取 YUV 数据的时间间隔。

4.5 大小码流录像示例

录像初始化函数，H264 的获取方式可参考 YUV 数据获取的流程, 已提供的参数为该场景下的固定值，未提供的参数请根据具体情况进行设置。

```
void awCamRecInit(String filename, int format, FileDescriptor fileDescriptor, int memLen, int
     offset, int width0, int height0, int frameRate0, int bitRate0, int width1, int height1,
    int frameRate1, int bitRate1, int audioMode, int audioSource, int audioSampleRate,int
    audioChannel, int timeLapse) {}
```

4.5.1 主码流录文件 ts + 子码流 h264 回调 + 录音

示例参数如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
- audioMode：AwCamRecAudioMode.NORMAL
- timeLapse：0
```

4.5.2 仅主码流录文件 ts + 录音（无子码流）

示例参数如下：

```
- filename：null
- width1：0
- height1：0
- framerate1：0
- bitrate1：0
- audioMode：AwCamRecAudioMode.NORMAL
- timeLapse：0
```

4.5.3 仅子码流 h264 回调 + 录音

示例参数如下：

```
- fileDescriptor：null
- memLen：0
- offset： 0
- width1：0
- height1：0
- framerate1：0
- bitrate1：0
- audioMode：AwCamRecAudioMode.NORMAL
- timeLapse：0
```

4.5.4 缩时录影

示例参数如下：

```
- timeLapse设置为大于0的整数
- 此功能会禁用录音功能
```

4.5.5 多通道录音

示例参数如下：

```
- audioChannel设置为 3~16之间的整数值
- 此功能需搭配void awCamRecSetAudioChannelIndex(int index)方法一起使用
```

场景示例：底层采用 TP2815 采集 16 路 MIC 数据，需要把 ch0/ch1/ch3 的数据分别封装到 3 路不同的视频文件中

第一步：分别调用 awCamRecInit 创建 3 个录像文件，这 3 个 awCamRecInit 函数中的 audioChannel 均需设置为 16；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第二步：分别调用 awCamRecSetAudioChannelIndex 为不同的录像文件指定不同的通道，若文件 1 为 ch0，则 index 为 0；

第三步：分别调用 awCamRecStart 开启 3 路录音录像，此时底层会自动拆分 16 通道的数据并编码封装至对应的录像文件中。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 FAQ

5.1 摄像头无法打开或打开后 hal 层报错 select timeout

第一步：先确认配置使用 awCamera 接口时是否配置的为 USE_CAMERA_HAL_AUTO_1_0 = true。

第二步：确认调用的 awCamera 接口是否正确，即是否为以下接口：

```
Camera open(int startId, int cameraNum, int width, int height)
Camera open(int startId, int cameraNum, int width, int height, int type)
```

第三步：检查/dev 目录下是否存在对应的设备节点，即/dev/videoX。

第四步：检查驱动是否正常，可查看 vin 的 debug 节点观察 frame ==> cnt 属性是否有一直在增长，如数值不变，则需排查驱动问题。

```
cat /sys/kernel/debug/mpp/vi
```

如果以上检查都正常，问题无法解决，请保存好 log 反馈给全志工程师解决。

5.2 打开摄像头预览出现黑屏现象

可以打开 CameraHal 中保存 yuv 原始帧的功能，进一步排查是否框架问题还是驱动问题。

第一步：修改 CameraHal auto_1_0 的代码，打开保存 yuv 原始帧的宏开关。

```
android/hardware/aw/camera/auto_1_0/CameraDebug.h
#define DBG_BUFFER_SAVE                1
```

第二步：模块编译 camerahal 的 so，并 push 到机器中。

第三步：adb shell 到机器目录中，创建/data/camera 目录，同时关闭 selinux 策略。

```
mkdir /data/camera
chmod 777 /data/camera
echo 0 > /sys/fs/selinux/enforce
```

第四步：复现问题，然后 pull 出/data/camera 下面的 yuv 数据，查看 yuv 是否正常，如显示黑屏那么定位为驱动问题，否则需要排查框架层问题。

如果以上检查都正常，问题无法解决，请保存好 log 反馈给全志工程师解决。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.3 预览/录像出现上/下边缘显示绿条

出现此问题的根本原因为没有对数据进行 16 位对齐，可采用如下方式解决：

```
预览绿条：
setprop persist.sys.camera.align "16"或
setprop persist.sys.camera.align "0"
录像绿条：
setprop persist.sys.camera.record.align "16"或
setprop persist.sys.camera.record.align "0"
```

每次设置之后需要重新打开 Camera 才能生效。

5.4 Camera Hal 是否支持 Hal 3.4

可通过方案配置文件 device_common.mk 中切换 hal 版本 hal3 支持双摄 android 标准接口

车载 Camera 不支持 hal3.4，目前车载 camera Hal 是基于 Hal 1.0 来开发的，流程相对简单。

5.5 如何后台抓取 log

连接 adb 或者通过串口，输入如下命令，然后在/data/media/awlog 目录可以看到相应的 log。

```
setprop persist.debug.logpersistd true
setprop persist.debug.logcat.enable true
setprop persist.debug.kernel_log.enable true
```

具体实现可参考 android/vendor/aw/public/package/bin/kmsgd。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。